Georgia Tech

ORDERING, WAITING, SCHED2

ALEXEY TUMANOV

School of Computer Science
faculty.cc.gatech.edu/~atumanov
atumanov@gatech.edu

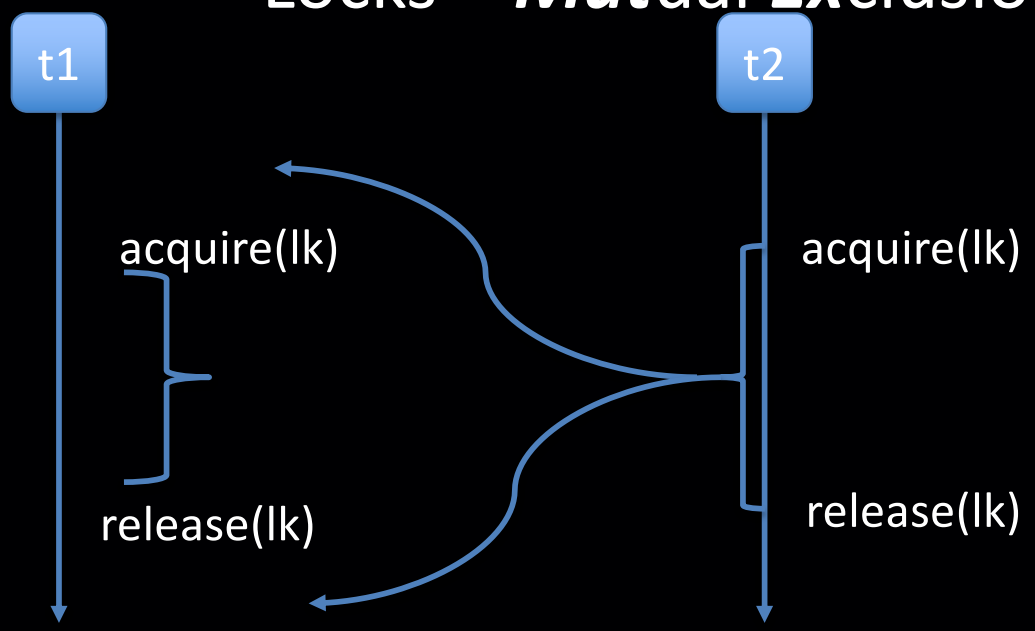
CREATING THE NEXT®

Georgia Tech

We've discussed locking, what does locking enforce?

(hint lock and mutex are synonymous)

Locks = ***Mut***ual ***Ex***clusion



DATA RACES

Do I need locks to avoid a **Data Race?**

Data Race:

Two *unordered* accesses to the same location in memory, at least one of which is a *write*.

What does "Unordered" mean?

Do locks add **ordering**?

acq(l)	acq(l)
a = x	a = y
rel(l)	rel(l)

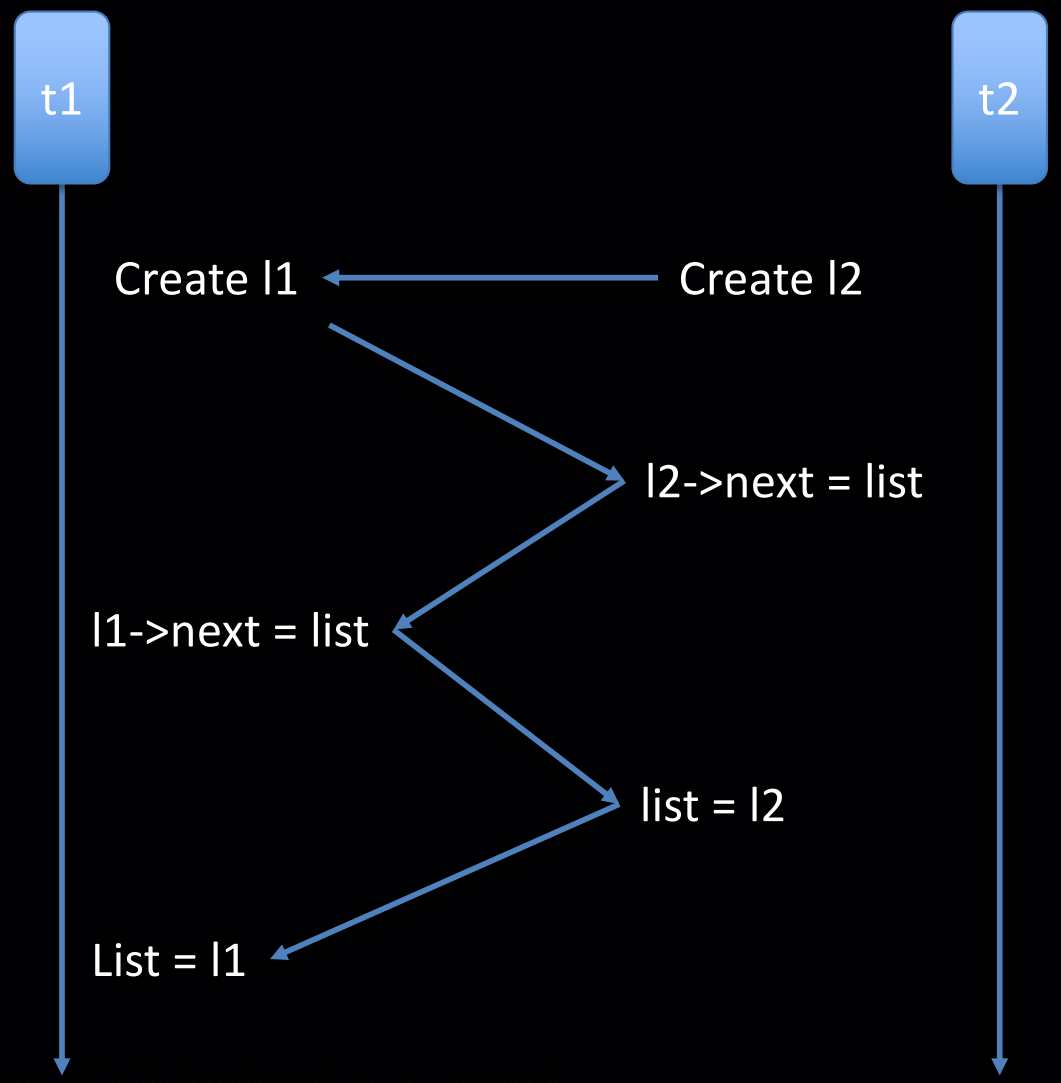
Locks provide us with an ordering *mechanism*…
Locks enable *dynamic* ordering
Locks reduce the *space of possibilities*

WEAK ORDERING SEMANTICS

- *Weak Ordering* (Relaxed Consistency)
  - Hardware, compiler can reorder reads and writes as long as the result is correct ***from the executing thread's perspective***!
  - Weak ordering does NOT concern itself with correctness ***across*** executing threads
  - Stores from one thread can appear *out of order* to another thread
  - A thread might see another thread's writes to different variables at different times
  - Without explicit synchronization, almost no ordering guarantees exist
- Lock acquire/release – a mechanism for *partial* order:
  - *Acquire* → all subsequent reads/writes in the cur thread happen ***after***
  - *Release* → all preceding reads/writes in the cur thread happen ***before***

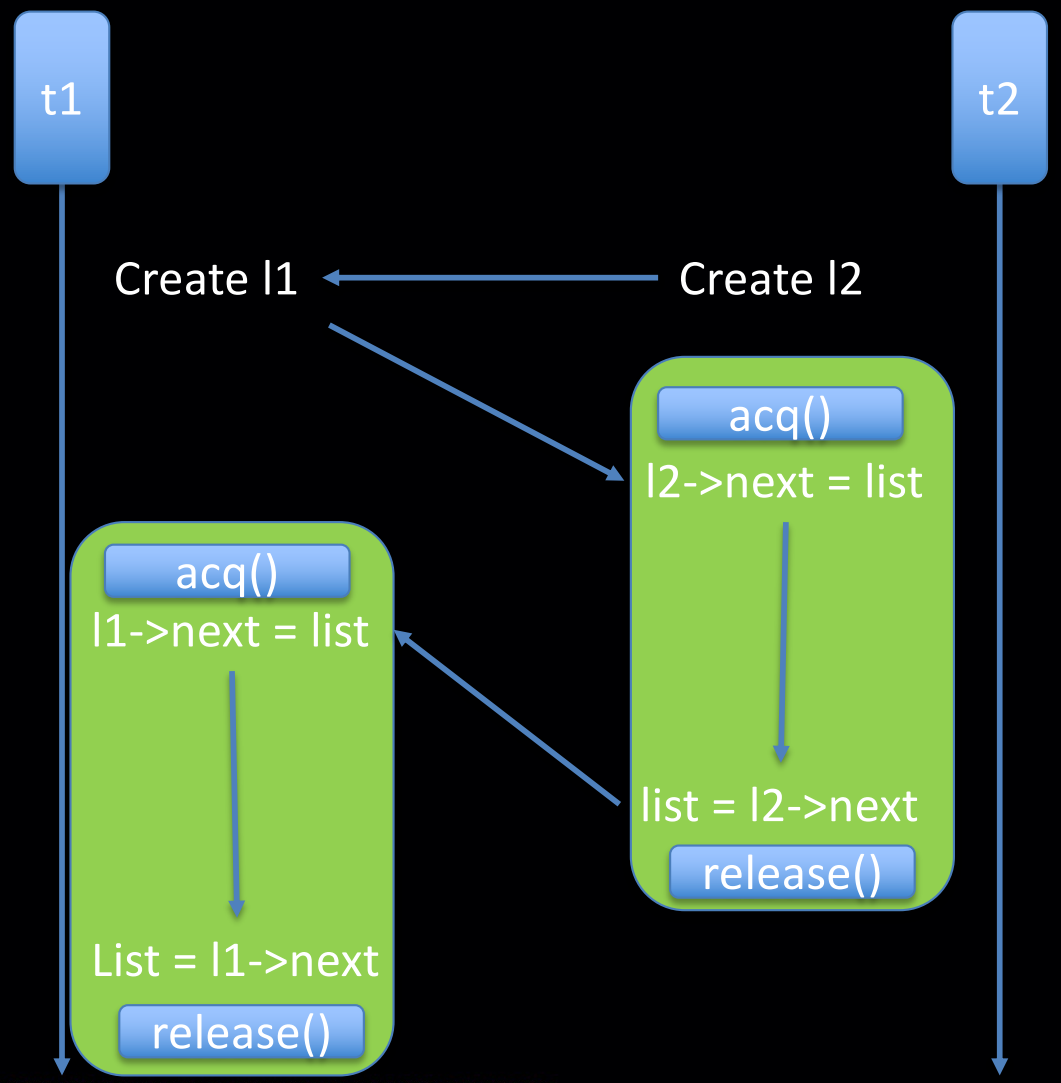
RACE CONDITION

```
struct list {
  int data;
  struct list *next;
};

struct list *list = 0;

void
insert(int data)
{
  struct list *l;

  l = malloc(sizeof *l);
  l->data = data;
  l->next = list;
  list = l;
}
```



RACE CONDITION

```
struct list {
  int data;
  struct list *next;
};

struct list *list = 0;

void
insert(int data)
{
  struct list *l;

  l = malloc(sizeof *l);
  l->data = data;
  l->next = list;
  list = l;
}
```



ORDERING SEMANTICS

- ***Unordered***:

Rt_1, Rt_2, Wt_1, Wt_2

Many possible interleavings

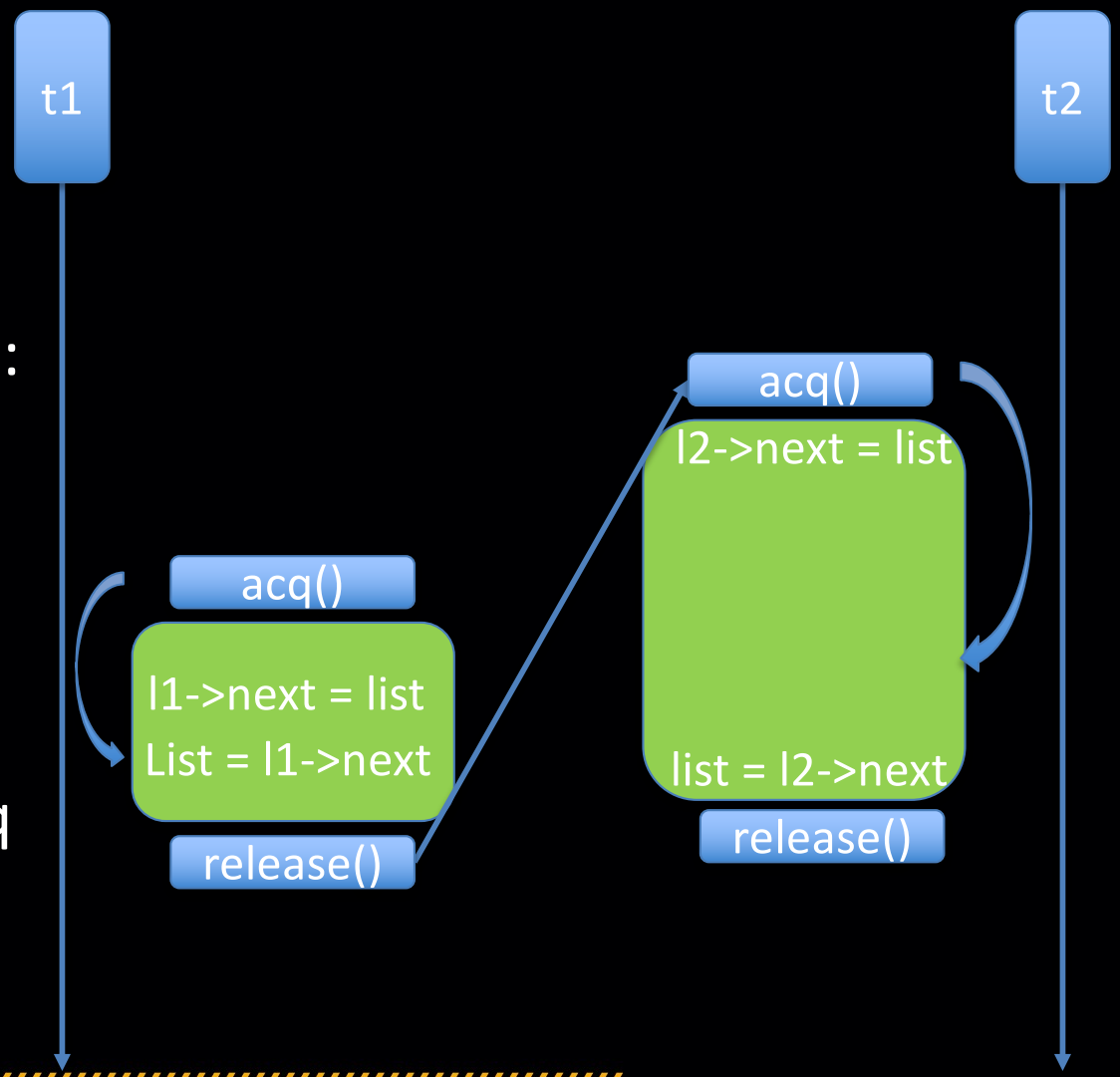
- ***Partially*** ordered (w/ locks):

Rt_1 Wt_1 Rt_2 Wt_2

Rt_2 Wt_2 Rt_1 Wt_1

- ***Fully*** ordered:

Exactly one globally visible seq



ORDERING SEMANTICS

- ***Unordered***:

Rt_1, Rt_2, Wt_1, Wt_2

Many possible interleavings

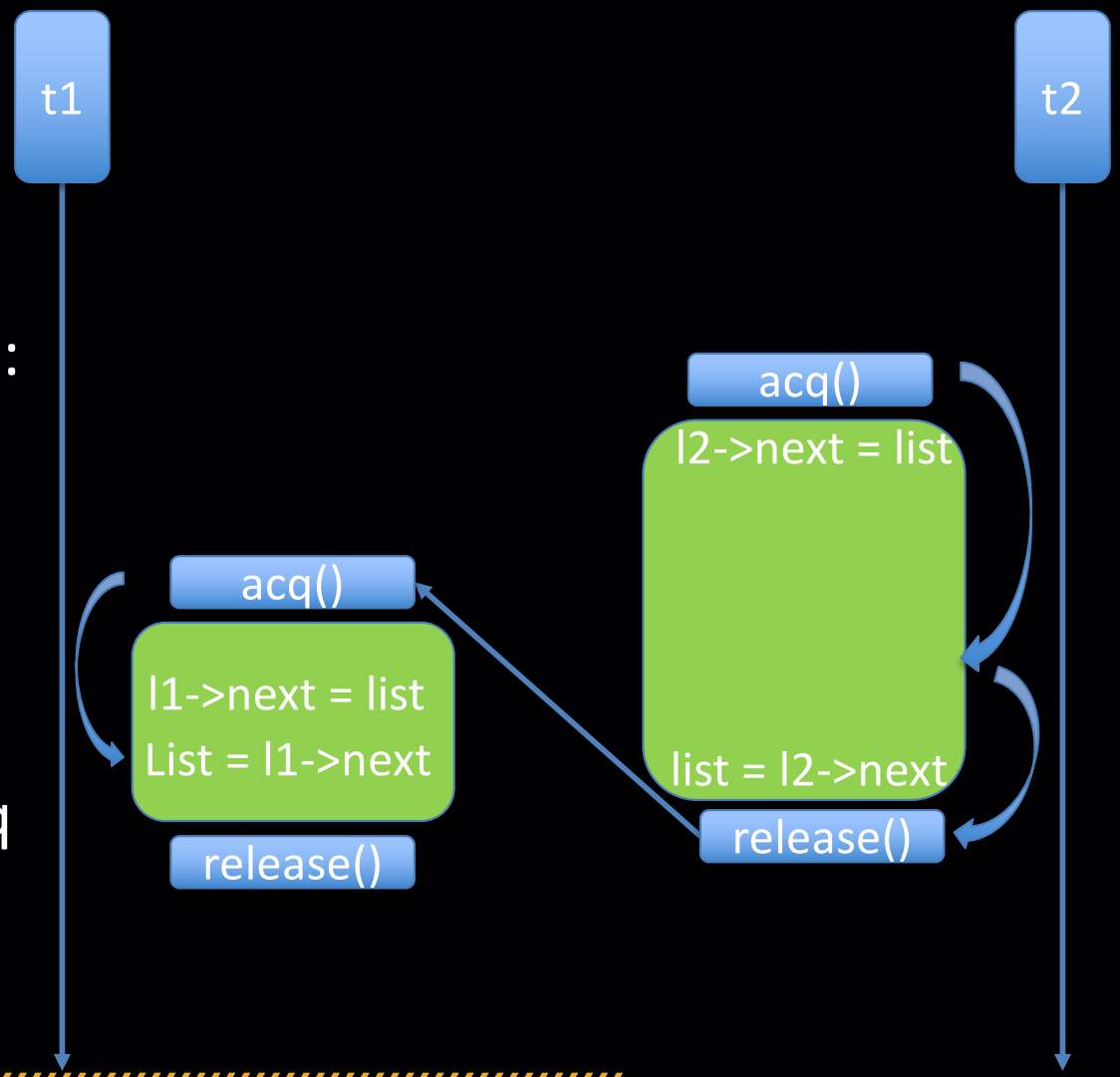
- ***Partially*** ordered (w/ locks):

Rt_1 Wt_1 Rt_2 Wt_2

Rt_2 Wt_2 Rt_1 Wt_1

- ***Fully*** ordered:

Exactly one globally visible seq



IS A MUTEX ENOUGH

*Are there other ways to add **ordering**?*

Think of an example: Try to build a producer-consumer queue:

- Producer:
  - Adds elements to the queue
- Consume:
  - As long as there are elements to take, remove them.
  - If there are no elements, they **wait** for the next element.
- ***Concept***: waiting
  - Notion of waiting can't be given by the mutex alone

safety

liveness

efficiency

IS A MUTEX ENOUGH

Think of an example: Try to build a producer-consume queue:

- Producer:
  - Adds elements to the queue
- Consume:
  - As long as there are elements to take, remove them.
  - If there are no elements, they **wait** for the next element.

I need some way to wait efficiently…

Goal:

- A computationally efficient waiting primitive
- Ordering primitive (P → C)

WAITING WITH ORDERING

Georgia Tech

Our main waiting primitive is a **Condition Variable (CV):**

- Allows a process to wait on a condition.
- Traditional interface (must know):
  - wait(lock *lk); -- sleeps until notified (may have spurious wakeups)
  - signal(); -- wakes up one waiter
  - broadcast(); -- wakes up all waiters

Georgia Tech

Can we write a producer/consumer using a CV and lock?

```
Produce(int v) {
    lock(lk)

    q.push()

    unlock(lk)
}
```

```
int consume() {
    lock(lk)

    v = q.pop()

    unlock(lk)
    return v;
}
```

- safety
- liveness
- efficiency

Are we ok? Are we done?

Georgia Tech

Can we write a producer/consumer using a CV and lock?

```
Produce(int v) {
  lock(lk)

  q.push()

  unlock(lk)
}
```

```
int consume() {
  lock(lk)

  if (!q.empty()):
     v = q.pop()

  unlock(lk)

  return v;
}
```

safety

liveness

efficiency

Are we ok? Are we done?

Georgia Tech

Can we write a producer/consumer using a CV and lock?

```
Produce(int v) {
  lock(lk)

  q.push()

  unlock(lk)

}
```

```
int consume() {
  lock(lk)
  if (q.empty()) {
    cv.wait(lk)
  }
  v = q.pop()
  unlock(lk)
  return v;
}
```

Are we ok? Are we done?

Georgia Tech

Can we write a producer/consumer using a CV and lock?

```
Produce(int v) {
  lock(lk)

  q.push()

  cv.signal()

  unlock(lk)

}
```

```
int consume() {
  lock(lk)

  if (q.empty()) {
    cv.wait(lk)
  }

  v = q.pop()
  unlock(lk)
  return v;

}
```

???

Are we ok? Are we done?

Georgia Tech

Can we write a producer/consumer using a CV and lock?

P1

```
Produce(int v) {
  lock(lk)

  q.push()
  cv.signal()
  unlock(lk)
}
```

P1: signal, unlock

C1

```
int consume() {
  lock(lk)
  if (q.empty()) {
    cv.wait(lk)
  }
  v = q.pop()
  unlock(lk)
  return v;
}
```

C1: wakeup

C2

```
int consume() {
  lock(lk)
  if (q.empty()) {
    cv.wait(lk)
  }
  v = q.pop()
  unlock(lk)
  return v;
}
```

C2: if, pop

time

Georgia Tech

Can we write a producer/consumer using a CV and lock?

```
Produce(int v) {
   lock(lk)

   q.push()

   cv.signal()

   unlock(lk)

}
```

```
int consume() {
   lock(lk)

   while (q.empty()) {
      cv.wait(lk)
   }

   v = q.pop()
   unlock(lk)
   return v;

}
```

Are we ok? Are we done?

WAITING

Note: when we wait, we must use a while loop, and unlock within the loop!

- This is due to race conditions! (not data races)
- Whenever you wait() it must be inside of a loop!!
  - (The c++11 condition_variable even forces this)

Georgia Tech

Where should we place the signal()?

```
Produce(int v) {
  lock(lk)
  q.push()
  cv.signal()
  unlock(lk)
}
```

```
Produce(int v) {
  lock(lk)
  q.push()
  unlock(lk)
  cv.signal()
}
```

```
Produce(int v) {
  cv.signal()
  lock(lk)
  q.push()
  unlock(lk)
}
```

```
int consume() {
  lock(lk)
  while (q.empty())
    cv.wait(lk)
  }
  v = q.pop()
  unlock(lk)
  return v;
}
```

Georgia Tech

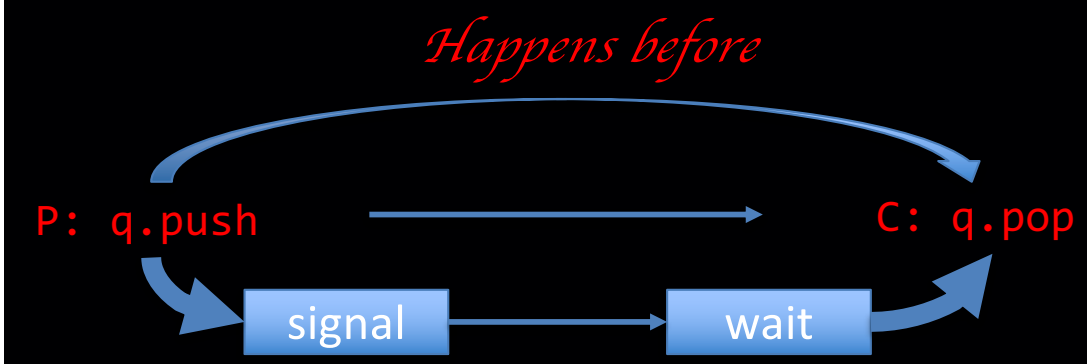
Where should we place the signal()?

```
Produce(int v) {
  lock(lk)
  q.push()
  cv.signal()
  unlock(lk)
}
```

```
Produce(int v) {
  lock(lk)
  q.push()
  unlock(lk)
  cv.signal()
}
```

```
Produce(int v) {
  cv.signal()
  lock(lk)
  q.push()
  unlock(lk)
}
```

```
int consume() {
  lock(lk)
  while (q.empty())
    cv.wait(lk)
  }
  v = q.pop()
  unlock(lk)
  return v;
}
```



WAITING MECHANISM

Georgia Tech

How **do we wait (mechanism)?**

- **Does user-space or the kernel switch the process?**
- **How do we handle switches within the kernel?**
- **How do we put processes to sleep?**
- **How do we wake sleeping processes?**

WAITING

Georgia Tech

What happens on the CPU when a process waits for another?

Can we use the CPU more productively in the meantime?

- **The CPU switches to running another process**
- **OR**
- **The CPU sleeps**

WAITING MECHANISM

What needs to be saved when sleeping?

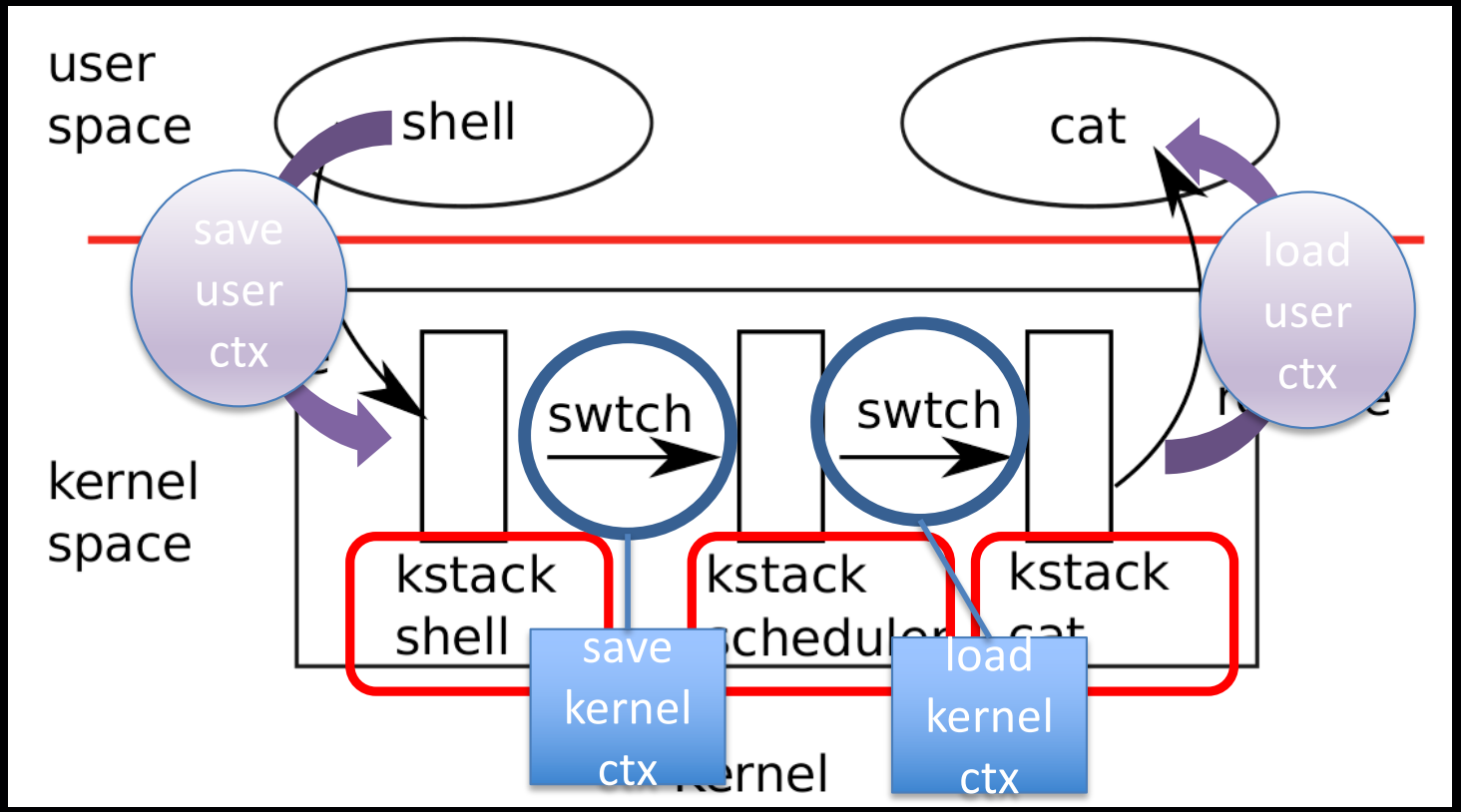
- User address space
- User context (regs)
- Kernel address space
- Kernel context (regs)

How do we save them?

SCHEDULING

$ cat

What happens when you sleep?



WAITING

What needs to be saved when sleeping?

- Address space – saved in proc's pagetable
- User context – saved on call to kernel
- Kernel context – needs to be saved explicitly!

How do we save the kernel context (reg state)?

xv6's swtch function!

WAITING

Georgia Tech

Xv6 swtch function:

- **Takes in two context structures**
- **Pushes all callee saved registers to the first**
- **Swaps to the new stack (from context structure)**
- **Pops all callee saved registers from the new stack**
  - **Note: new stack's return address!**
- **Returns (using the new stack's return address)**

XV6 SWTCH

```
.globl swtch
swtch:
  movl 4(%esp), %eax
  movl 8(%esp), %edx

  # Save old callee-saved registers
  pushl %ebp
  pushl %ebx
  pushl %esi
  pushl %edi

  # Switch stacks
  movl %esp, (%eax)
  movl %edx, %esp

  # Load new callee-saved registers
  popl %edi
  popl %esi
  popl %ebx
  popl %ebp
  ret
```

old

new

old

new

QUESTION

Why does swtch need to only save/restore callee saved registers?

- Swtch → is a *voluntary* preemption mechanism (cooperative)

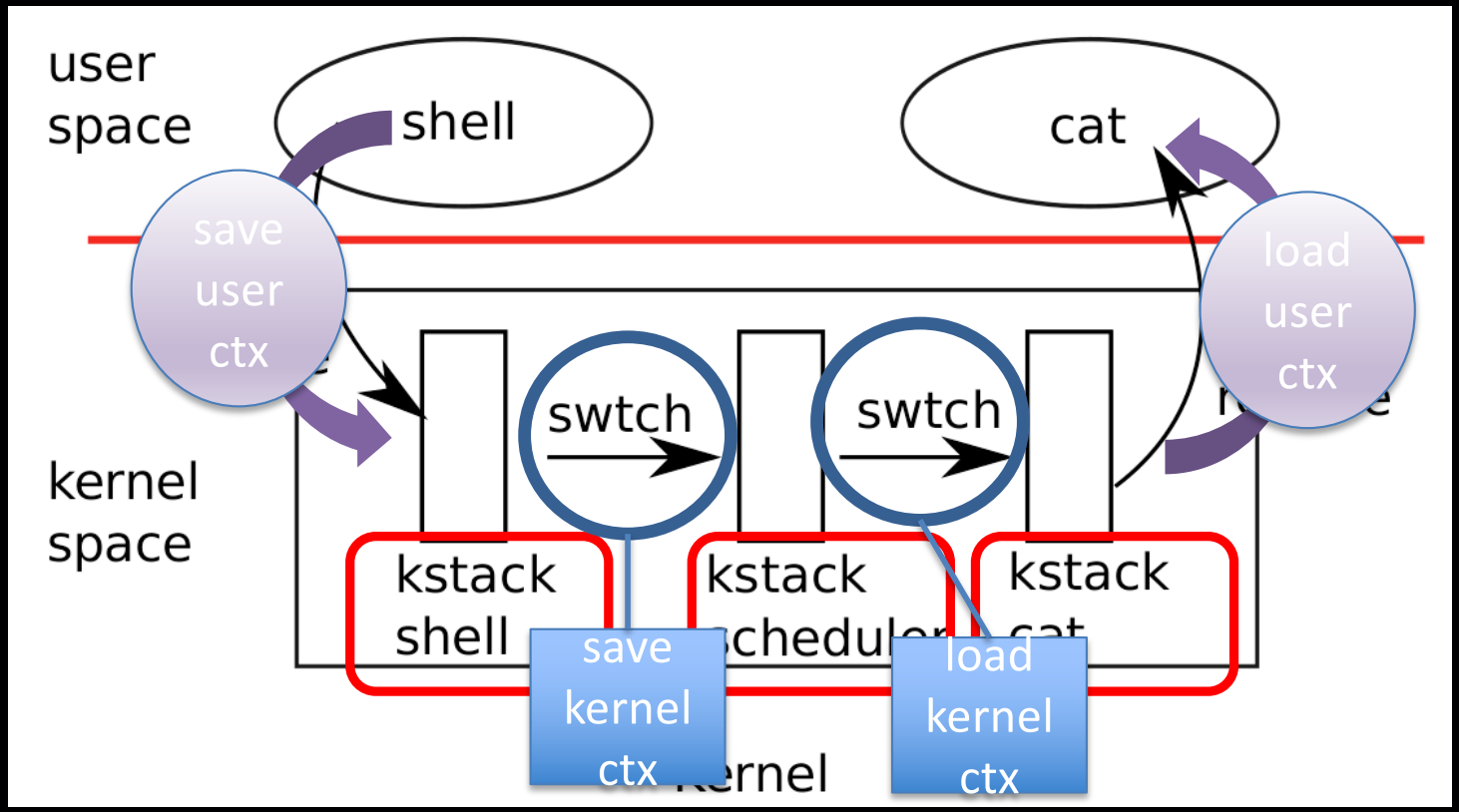
Does the trap handler need to only save/restore callee saved registers?

trap handler runs on an interrupt, not cooperative

caller didn't save anything before ending up in the handler

KERNEL CONTEXT SWITCHING

$ cat



Georgia Tech

Where does a process go when it waits?

- Traditionally, use a wait list:
  - Once process sleeps, put it on a wait list
  - Can wake up processes from that waitlist when needed
  - NOTE: xv6 use
    processes, the
    - It allows
      (chan va
    - See “wakeup1” and “sleep”

```
wakeup1(void *chan)
{
  struct proc *p;

  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    if(p->state == SLEEPING && p->chan == chan)
      p->state = RUNNABLE;
}
```

Georgia Tech

So, can we use this to build a smarter lock?

- What would a better version of spinlock look like?
  - Sleep when lock is currently held by other
  - Wake up a sleeper on lock release

sleeplock()

WAITING

Lets build a lock function:

Struct lock { struct spinlock *l, int locked; List waitlist; };

Void lock (struct lock *lk) {

```
acquire(lk->l);
while (locked == 1) {
  sleep(&lk->waitlist, l);
}
locked = 1;
release(lk);
```

}

WAITING

Lets build an unlock function:

Struct lock { struct spinlock sp; int lock; List waitlist; };

Void unlock (struct lock *lk) {

```
acquire(lk);
lk->locked = 0;
wake_one(&lk->waitlist);
release(lk);
```

}

SCHEDULING

Scheduling policies in Linux

SCHED_OTHER: normal time/schedule sharing (default: CFS)

SCHED_IDLE: Very low prioritized processes

SCHED_BATCH: CPU intensive processes

SCHED_FIFO: first in, first out, real time processes

SCHED_RR: round robin real time processes

XV6 SCHEDULER CODE

```
void
scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();
  c->proc = 0;

  for(;;){
    // Enable interrupts on this processor.
    sti();

    // Loop over process table looking for process to run.
    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != RUNNABLE)
        continue;

      // Switch to chosen process.  It is the process's job
      // to release ptable.lock and then reacquire it
      // before jumping back to us.
      c->proc = p;
      switchuvm(p);
      p->state = RUNNING;

      swtch(&(c->scheduler), p->context);
      switchkvm();

      // Process is done running for now.
      // It should have changed its p->state before coming back.
      c->proc = 0;
    }
    release(&ptable.lock);

  }
}
```

CREATING THE NEXT®

XV6 SCHEDULER

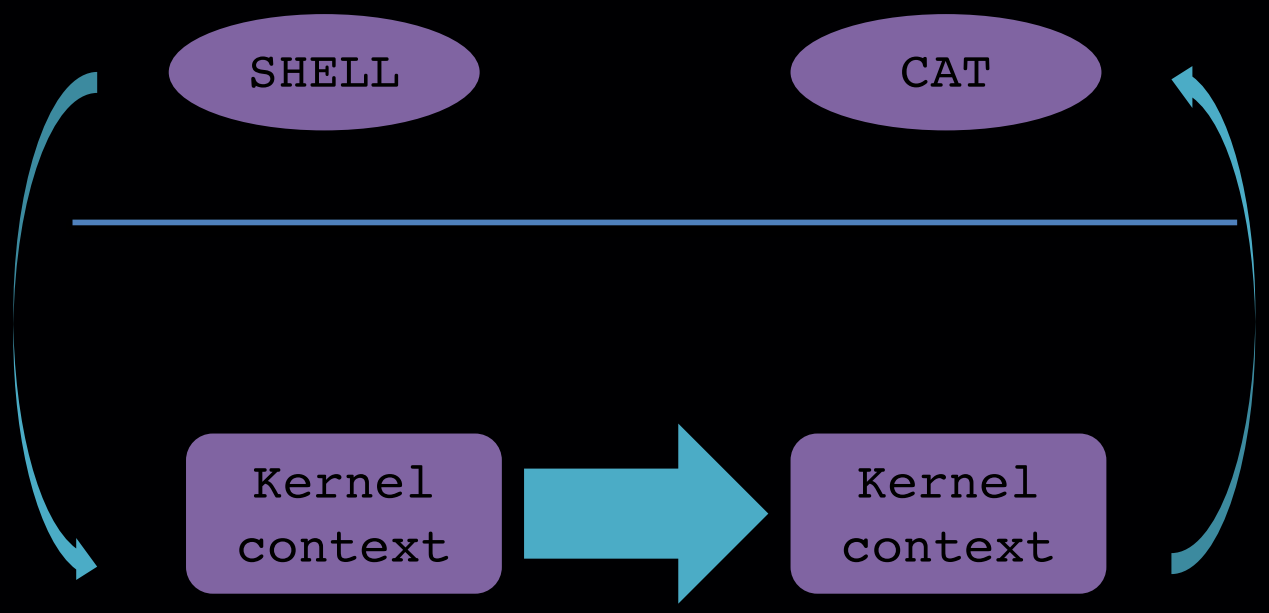
xv6 void scheduler(void)

```
// Loop over process table looking for process to run.
acquire(&ptable.lock);
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
  if(p->state != RUNNABLE)
    continue;

  // Switch to chosen process.  It is the process's job
  // to release ptable.lock and then reacquire it
  // before jumping back to us.
  proc = p;
  switchuvm(p);
  p->state = RUNNING;
  swtch(&cpu->scheduler, p->context);
  switchkvm();

  // Process is done running for now.
  // It should have changed its p->state before coming back.
  proc = 0;
}
release(&ptable.lock);
```

```
// Switch h/w page table register to the kernel-only page table,
// for when no process is running.
void
switchkvm(void)
{
  lcr3(V2P(kpgdir));   // switch to the kernel page table
}
```

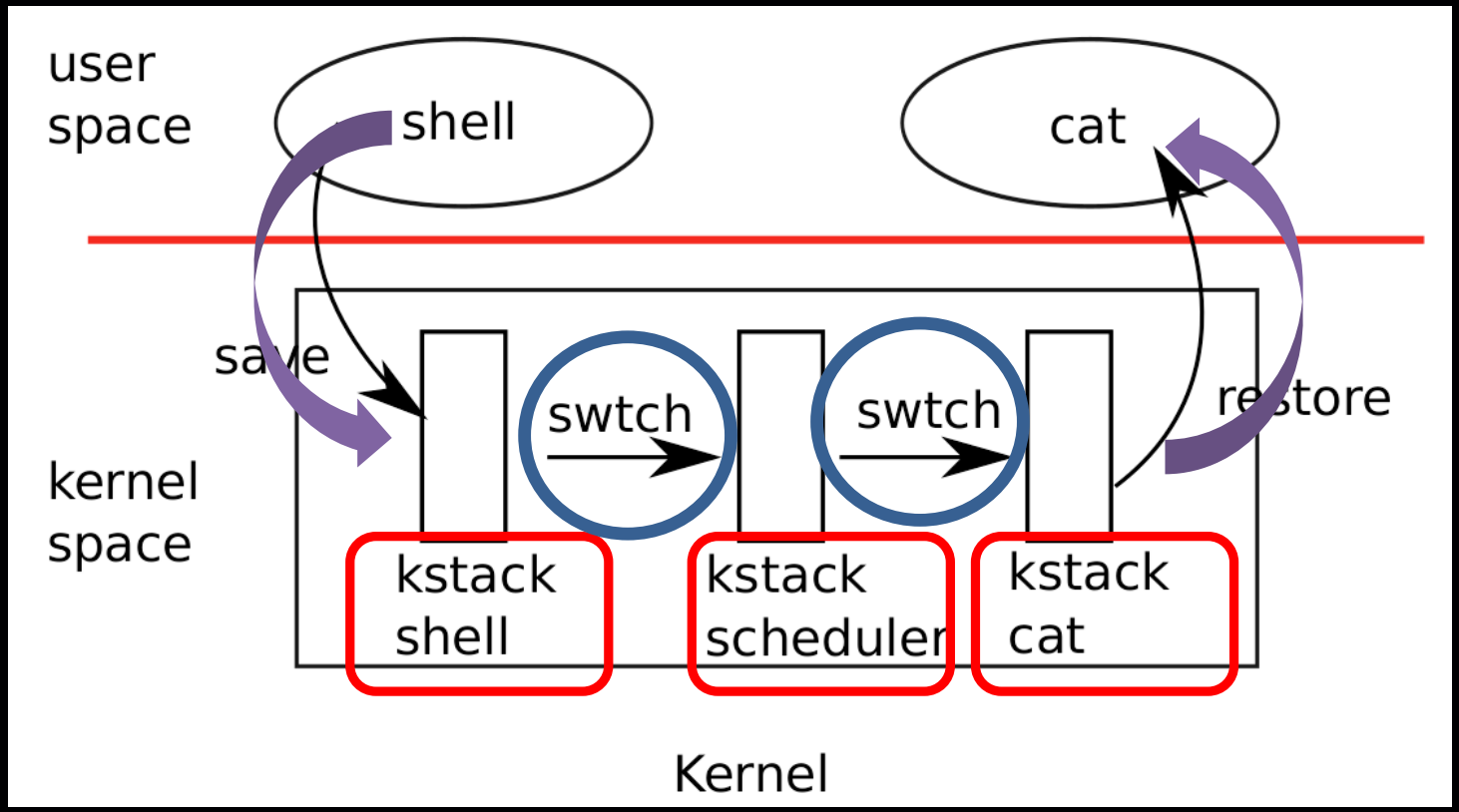
SCHEDULING: CONTEXT SWITCH

Big picture: This is what we *want* to do



Georgia Tech

Big picture: this is what we *have* to do



SCHEDULING SUMMARY

```
void
yield(void)
{
  acquire(&ptable.lock);  //DOC: yieldlock
  myproc()->state = RUNNABLE;
  sched();
  release(&ptable.lock);
}
```

Context Switching: Implementation Challenges

How to switch from one process to another?

- Context switching

How to make context switching transparent?

- Timer interrupts (if timer interrupt: yield)

How to switch among processes running concurrently?

- Locking

How to coordinate processes?

- Sleep on events (e.g., pipe, child exit)
- Shared memory (with threading implemented!)

Georgia Tech

Two kinds of context switch

From a process's kernel context to CPU scheduler context

From the scheduler context to a process's kernel context.

xv6 **never** directly switches from *user-space to user-space*

- user-kernel transition (system call or interrupt, incl. timer interrupt)
- context switch to scheduler
- context switch to new process's kernel thread
- trap return

SCHEDULING: CONTEXT SWITCHING

Context switching

- Every xv6 process has its own kernel context:

  kernel stack and register set (incl. %esp, %eip)

- Every CPU has its own scheduler thread
- Switching from one thread to another
  - Save and load CPU registers, including %esp, %eip.

SCHEDULING: SWTCH

swtch

```
void swtch(struct context**, struct context*);
```

Doesn't know about threads just saves and loads sets of registers called contexts

When time to give up CPU, kernel thread calls swtch to save itself and return to scheduler context

Context is a struct context*, stored on the kernel stack

CPU pushed onto stack and saves stack pointer to *old

Copies new to %esp, pops previous registers, and returns

yield

At the end of each interrupt, trap can call yield

yield -> sched -> swtch

Switches from proc=>context to cpu=>scheduler

```
void
yield(void)
{
  acquire(&ptable.lock);  //DOC: yieldlock
  myproc()->state = RUNNABLE;
  sched();
  release(&ptable.lock);
}
```

Georgia Tech

Scheduling

Process giving up the CPU must

- acquire ptable.lock
- release any other locks (remember deadlocks)
- update its own state (e.g., RUNNABLE, SLEEPING)
- call sched

yield, *sleep*, and *exit* all do these steps:

- Exit: I'm done, who's next
- Sleep: I have nothing to do, you go
- Yield : voluntary/cooperative

SCHEDULING

sched and scheduler

Kernel thread always gives up in sched and switches to same location in scheduler

Almost always switches to a process in sched.

Thread switches follow a simple pattern between sched and scheduler

- Coroutines

Exception is forkret when process is first sched

```
void
sched(void)
{
  int intena;
  struct proc *p = myproc();

  if(!holding(&ptable.lock))
    panic("sched ptable.lock");
  if(mycpu()->ncli != 1)
    panic("sched locks");
  if(p->state == RUNNING)
    panic("sched running");
  if(readeflags()&FL_IF)
    panic("sched interruptible");
  intena = mycpu()->intena;
  swtch(&p->context, mycpu()->scheduler);
  mycpu()->intena = intena;
}
```

Georgia Tech

Scheduler

Loops over process table looking for RUNNABLE processes

Finding one, sets current per-CPU process to proc

Switches page table with switchuvm, marks as RUNNING, and calls swtch

Georgia Tech

swtch

void swtch(struct context**, struct context*);

Doesn't know about threads just saves and loads sets of registers called contexts

When time to give up CPU, kernel thread calls swtch to save itself and return to scheduler context

Context is a struct context*, stored on the kernel stack

CPU pushed onto stack and saves stack pointer to *old

Copies new to %esp, pops previous registers, and returns

yield

At the end of each interrupt, trap can call yield

yield -> sched -> swtch -> scheduler() -> swtch

Switches from proc=>context to cpu=>scheduler